

Aplicación de los Principios SOLID en Java

Refactorización de un proyecto base aplicando los cinco principios SOLID.

INFORMACIÓN DE LA ASIGNACIÓN

Categoría de grupo	Semana 1 - Autoinscripcion
Nombre de grupo	S1 1 - Martes
Integrantes	Josue Riera · Joao Conde · Victor Suquilanda
Lenguaje / Entorno	Java (OpenJDK) · VS Code

Objetivos de la actividad

- Comprender y aplicar los principios SOLID en la práctica.
- Refactorizar un proyecto en Java para mejorar su diseño y calidad.
- Reflexionar sobre el impacto de SOLID en el desarrollo de software.

Link del repositorio

HTTPS → <https://github.com/Lyriom/calidad32>
SSH → `git@github.com:Lyriom/calidad32.git`

Cada principio está en su propio paquete (`srp` · `ocp` · `lsp` · `isp` · `dip`), cada uno con su clase `Main`. El código fuente, las capturas y las reflexiones están en el repositorio.

Ejecución

1. SRP — Single Responsibility Principle

Sistema de nómina: cada clase tiene una sola responsabilidad (datos, cálculo, formato, persistencia) y ServicioNomina solo coordina.

```
SRP - Single Responsibility Principle

$ java -cp target/classes srp.Main
=== Rol de pagos: empleado 1 ===
Guardando rol de pagos en la base de datos...
> Empleado: Josue Riera          | Cargo: Backend Developer    | Bruto: $ 1500.00 | Neto a pagar: $ 1366.25

=== Rol de pagos: empleado 2 ===
Guardando rol de pagos en la base de datos...
> Empleado: Joao Conde          | Cargo: QA Engineer         | Bruto: $ 1200.00 | Neto a pagar: $ 1053.00

=== Rol de pagos: empleado 3 ===
Guardando rol de pagos en la base de datos...
> Empleado: Victor Suquilanda   | Cargo: Frontend Developer   | Bruto: $ 1350.00 | Neto a pagar: $ 1209.63
```

2. OCP — Open/Closed Principle

Cálculo de descuentos por estrategia. Se agrega DescuentoTemporada sin modificar CalculadoraPrecio: abierto a extensión, cerrado a modificación.

```
OCP - Open/Closed Principle

$ java -cp target/classes ocp.Main
Monto base de la compra: $200.0

Cliente Regular (0%)          -> Total a pagar: $200.00
Cliente Premium (10%)         -> Total a pagar: $180.00
Cliente VIP (20%)             -> Total a pagar: $160.00
Promo Temporada (35%)         -> Total a pagar: $130.00
```

3. LSP — Liskov Substitution Principle

Figuras geométricas: cualquier subtipo de Figura sustituye a otro en el cálculo de área total sin romper el programa.

```
LSP - Liskov Substitution Principle

$ java -cp target/classes lsp.Main
--- Calculando area y perimetro de cada figura ---
Circulo          | area =    78.54 | perimetro =   31.42
Rectangulo       | area =    24.00 | perimetro =   20.00
Cuadrado         | area =     9.00 | perimetro =   12.00
Triangulo equilatero | area =     6.93 | perimetro =   12.00

Area total de todas las figuras: 118.47
```

4. ISP — Interface Segregation Principle

Dispositivos de oficina: interfaces pequeñas (Impresora, Escaner, Fax). La impresora básica solo implementa lo que usa.

```
ISP - Interface Segregation Principle

$ java -cp target/classes isp.Main
--- Impresora Multifuncion ---
Multifuncion: imprimiendo 'Informe_SOLID.pdf'
Multifuncion: escaneando 'Contrato.png'
Multifuncion: enviando fax de 'Factura.pdf' a +593 99 123 4567

--- Impresora Basica ---
Basica: imprimiendo 'Ticket.txt'
```

5. DIP — Dependency Inversion Principle

El Interruptor (alto nivel) depende de la abstracción Dispositivo; la implementación concreta se inyecta por constructor.

DIP - Dependency Inversion Principle

```
$ java -cp target/classes dip.Main
--- Interruptor conectado a una Bombilla ---
Bombilla encendida.
Bombilla apagada.

--- Interruptor conectado a un Ventilador ---
Ventilador girando a velocidad media.
Ventilador detenido.

--- Interruptor conectado a una Cafetera ---
Cafetera preparando cafe.
Cafetera en reposo.
```

Reflexiones

La reflexión completa por cada principio está en el archivo README.md del repositorio. A continuación las reflexiones individuales del equipo:

Josue Riera. El principio que más me costó fue **DIP**. Al inicio no veía el problema de que el Interruptor creara su propio dispositivo adentro; recién al querer cambiar la bombilla por un ventilador entendí que tocaba modificar una clase que ya funcionaba. Inyectar la dependencia por constructor lo resolvió y dejó el código más fácil de probar. En proyectos reales aplicaría DIP siempre que dependa de servicios externos: definir una interfaz e inyectar la implementación desde afuera.

Joao Conde. El más desafiante fue **LSP**, porque entender que la herencia es un contrato de comportamiento (y no solo reutilizar código) me tomó tiempo. Con las figuras quedó claro: si un subtipo no devuelve un área válida,

el cálculo total se cae. SOLID reduce las sorpresas y hace el código más predecible; como QA, es justo lo que facilita el testing.

Victor Suquilanda. El que más trabajo me dio fue **SRP**: no es difícil de entender, pero sí de aplicar bien, porque cuesta decidir dónde termina una responsabilidad y empieza otra. Separar la nómina en varias clases pequeñas la dejó mucho más mantenible. SOLID también ordena el trabajo en equipo: cada quien toca una clase distinta sin pisarse.